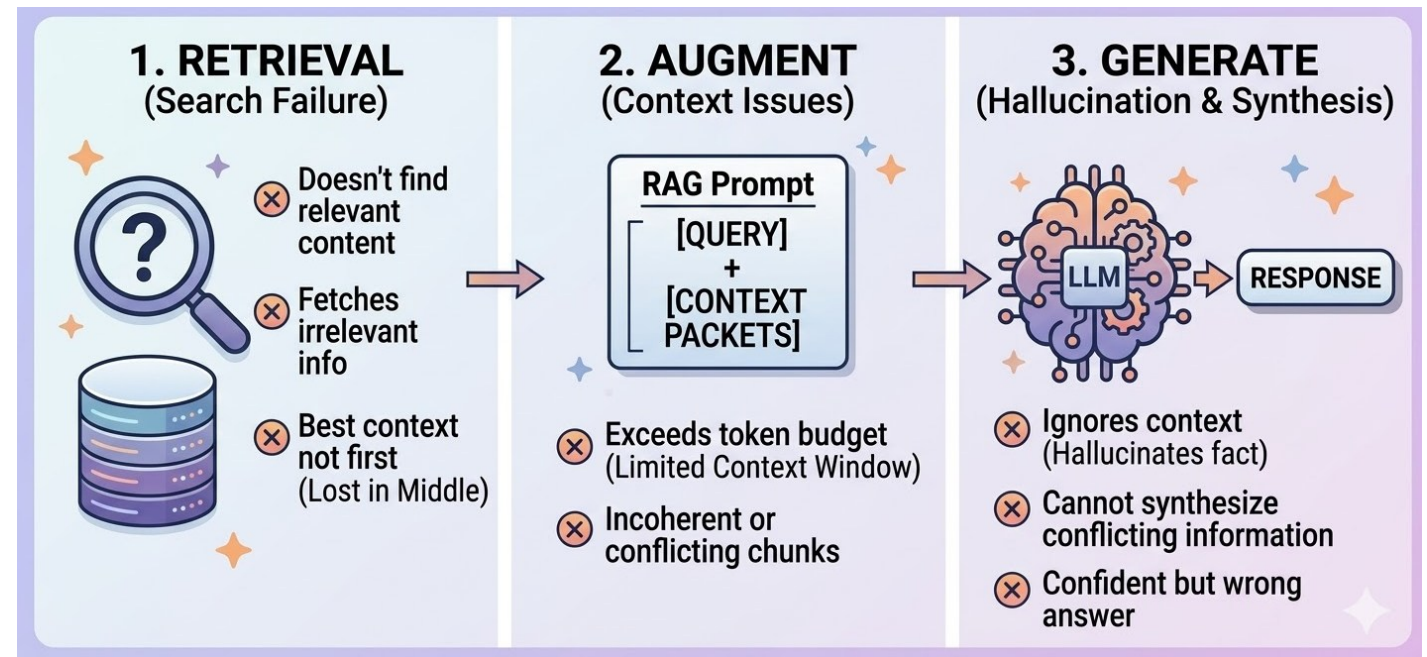


CHALLENGES IN CONVENTIONAL RAG

The basic RAG pipeline retrieves chunks, stuffs them into a prompt, generate an answer

It works surprisingly well for simple use cases, but production systems face harder problems:

- Vague queries that retrieve the wrong context
- Context windows that overflow with marginally relevant text
- Questions that span images and tables
- The need to measure whether any of it is actually working



UNDERSPECIFIED QUERIES

- Users rarely write perfect search queries
- Real-world queries are underspecified
- They are short, ambiguous, and miss key terms that would help the retriever find the right documents
- When the query is vague, the embedding vector is vague too
- The embedding vector sits in a generic region of the vector space and retrieves generic, unfocused chunks
- The result: the LLM gets poor context and produces a poor answer

User types:

"pricing"

"it's not working"

"tell me about RAG"

What they actually mean:

"What are the subscription pricing tiers for the enterprise plan in 2024?"

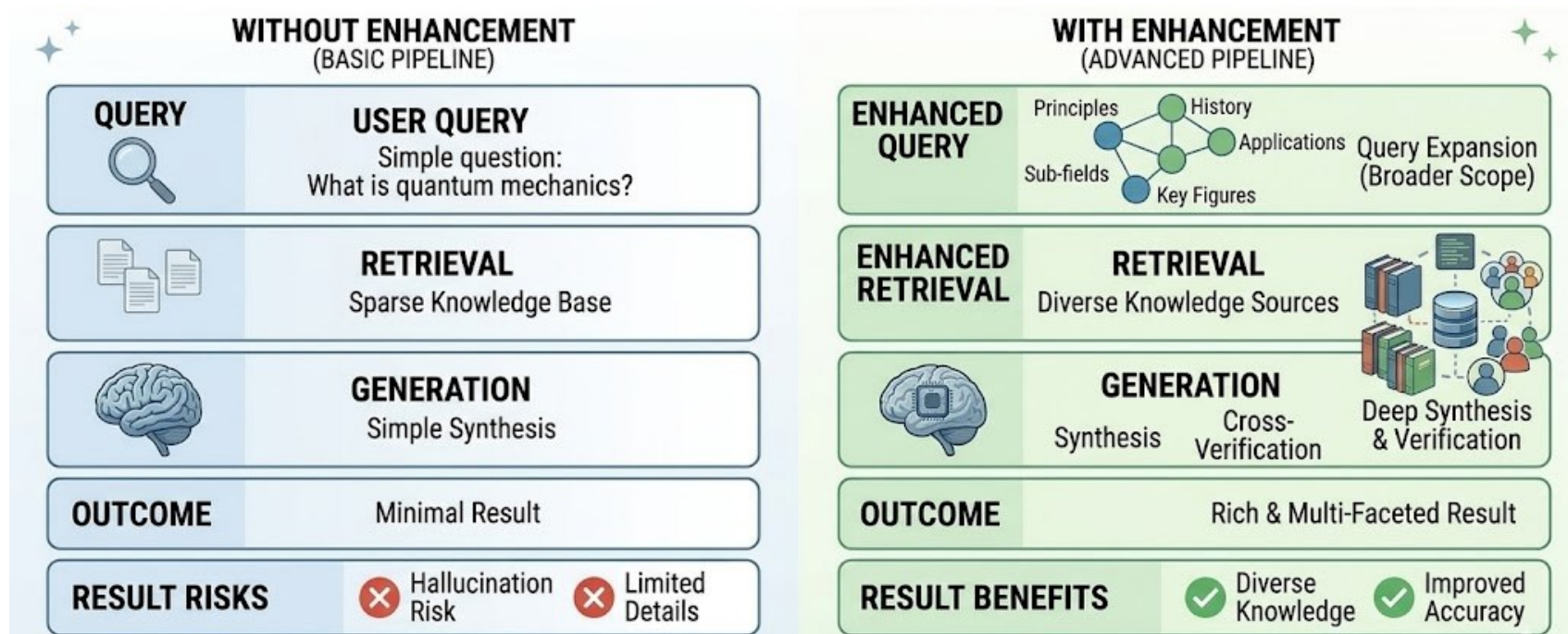
"The OAuth token refresh endpoint returns a 401 error after upgrading to v3.2"

"Explain the retrieve-augment-generate pipeline, including chunking strategies and re-ranking"

QUERY EXPANSION

Query expansion enriches the original query with additional terms, synonyms, or related concepts before sending it to the retriever

The goal is to broaden recall to catch relevant documents that the original query's embedding would have missed



Let's learn about some query expansion methods

MULTI-QUERY EXPANSION

Use the LLM itself to generate multiple reformulations of the user's query, retrieve results for each, and merge them

This works because different phrasings activate different regions of the embedding space, improving the chance that at least one variant lands near the right documents

```
Original query:  
"How does caching work?"
```

```
LLM generates 3 variants:
```

1. "What caching strategies are used to reduce latency in production systems?"
2. "How are frequently accessed results stored to avoid redundant computation?"
3. "What is the difference between response caching and retrieval caching?"

```
Each variant → independent retrieval → union of results (deduplicated)
```

- **Use** multi-query expansion when user queries are ambiguous, brief, or use different terminology than the source documents
- This can also be useful when the relevant context for a query is split across multiple documents; each query variant could focus on retrieving one chunk
- **Avoid** it for straightforward, fact-based queries or latency-sensitive applications, as the multiple LLM-driven searches significantly increase API costs

QUERY REWRITING

Instead of generating multiple queries, rewrite the original into a single, better-specified version
This is especially useful in conversational settings where the query depends on prior turns

Conversation:

User: "What is RAG?"

Assistant: "RAG stands for Retrieval-Augmented Generation..."

User: "How does the retrieval part work?"

↑ "*the retrieval part*" is ambiguous without context

Rewritten query:

"How does the retrieval component of a Retrieval-Augmented Generation (RAG) system work?"

The rewriter (usually an LLM) resolves coreferences, expands abbreviations,
and adds context from the conversation history

Avoid it when dealing with high-volume, low-latency applications where the extra LLM call would introduce unacceptable cost or delays, or when the original queries are already precise and keyword-dense

HYPOTHETICAL DOCUMENT EMBEDDING (HyDE)

A creative approach: instead of expanding the query, ask the LLM to generate a hypothetical answer to the query, then embed that hypothetical answer and use it as the search vector

The intuition: a hypothetical answer is closer in embedding space to real answers than the short query is

Query: "How does photosynthesis work?"

1. LLM generates a hypothetical answer (need not be perfect):

"Photosynthesis is the process by which plants convert sunlight, water, and carbon dioxide into glucose and oxygen using chlorophyll in their chloroplasts..."

2. Embed the hypothetical answer (not the query)

3. Search the vector store with this embedding

Use HyDE when dealing with a "cold start" scenarios where direct vector similarity fails to find relevant chunks due to a lack of overlapping keywords

Avoid it in high-precision or domain-specific tasks where the hypothetical document might contain factual hallucinations that steer the retrieval toward convincing but irrelevant misinformation

STEP-BACK PROMPTING

For highly specific queries, step back to a more general question first, retrieve broader context, then answer the specific question

Original query:

"What was the GDP of France in Q3 2024?"

Step-back query:

"What are the recent economic indicators and GDP trends for France?"

→ Retrieve broader economic context

→ Then the LLM can locate the specific Q3 2024 figure within that context

Use step-back prompting when a query is too specific to find direct matches and requires retrieving high-level principles or broader context

Avoid it for factual, "needle-in-a-haystack" queries where the specific details are exactly what is needed for the answer.

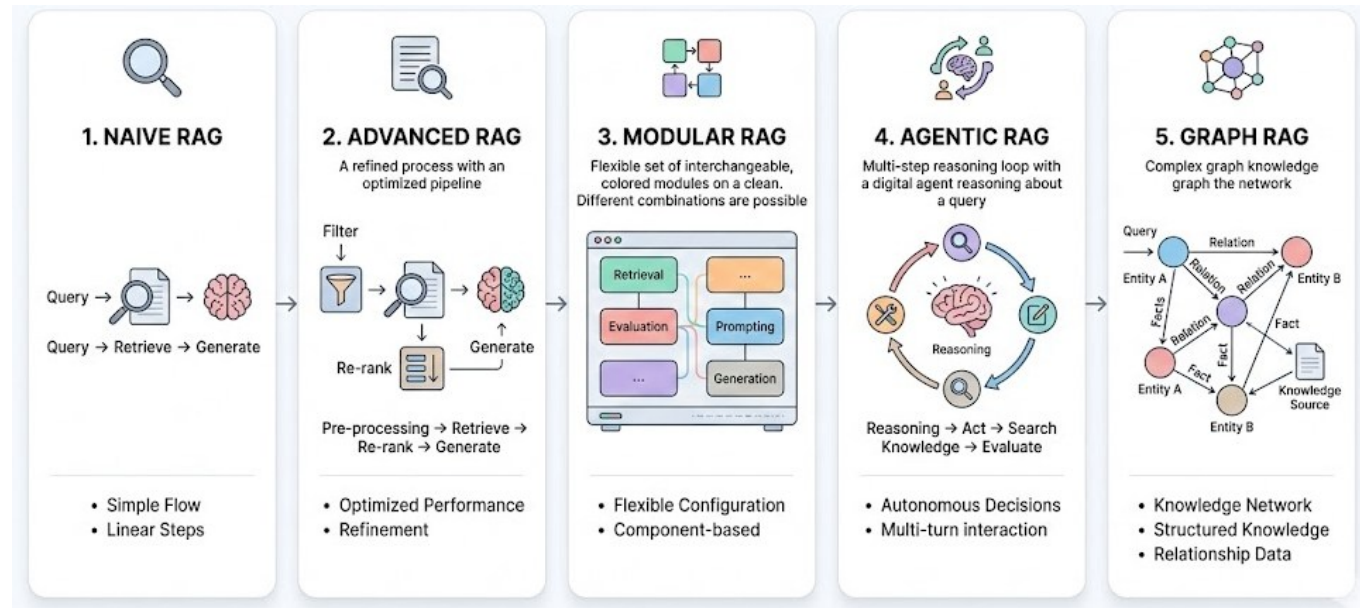
WHEN EXPANSION HELPS

Scenario	Expansion Helps?	Why
Short, vague queries ("pricing", "error")	Yes	Expansion adds specificity and retrieves diverse relevant chunks
Conversational follow-ups ("what about that?")	Yes	Rewriting resolves coreferences and adds context
Domain-specific jargon the user doesn't know	Yes	LLM can expand into domain terms
Already well-specified queries	No / Risky	Expansion may introduce tangential terms that dilute retrieval
Factoid queries with exact answers	No / Risky	Extra variants may retrieve contradictory information
Very large corpora with noisy data	Risky	More queries = more results = more noise to filter

Best practice: Use expansion selectively. Apply it when initial retrieval quality is low (measured by hit rate,) and always follow up with re-ranking or filtering to remove noise that expansion may introduce

TYPES OF RAG FRAMEWORKS

- RAG has evolved rapidly from a simple three-step pipeline into a family of architectures, each suited to different complexity levels
- Understanding these types helps you choose the right design for your use case:
 - Naive RAG
 - Advanced RAG
 - Modular RAG
 - Agentic RAG
 - Graph RAG



NAIVE RAG

Naïve RAG is the simplest implementation and the starting point for most projects

How it works: Embed the query, retrieve the top-K chunks from a vector store, concatenate them into the prompt, and generate

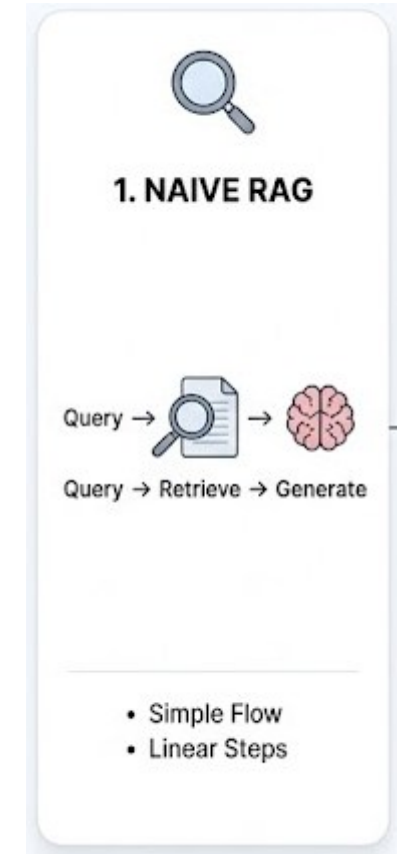
Strengths:

- Simple to implement (< 50 lines of code)
- Fast iteration and prototyping
- Good enough for focused, single-domain Q&A

Weaknesses:

- No query optimisation garbage in, garbage out
- No retrieval refinement relies entirely on embedding quality
- Chunks may be redundant, irrelevant, or poorly ordered
- No feedback loop, errors are invisible

Best for: Prototypes, demos, internal tools with a small, clean knowledge base



ADVANCE RAG

Advance RAG adds pre-retrieval and post-retrieval optimisation stages around the core pipeline

What it adds over Naive RAG:

Stage	Technique	Purpose
Pre-retrieval	Query expansion, HyDE, step-back prompting	Improve recall
Pre-retrieval	Metadata filtering	Narrow search space
Post-retrieval	Cross-encoder re-ranking	Improve precision
Post-retrieval	Context compression / summarisation	Fit more signal in fewer tokens
Post-retrieval	Chunk reordering (most relevant first & last)	Mitigate "lost in the middle"

Best for: Production systems where retrieval quality directly impacts user trust
(enterprise search, customer support, compliance Q&A)

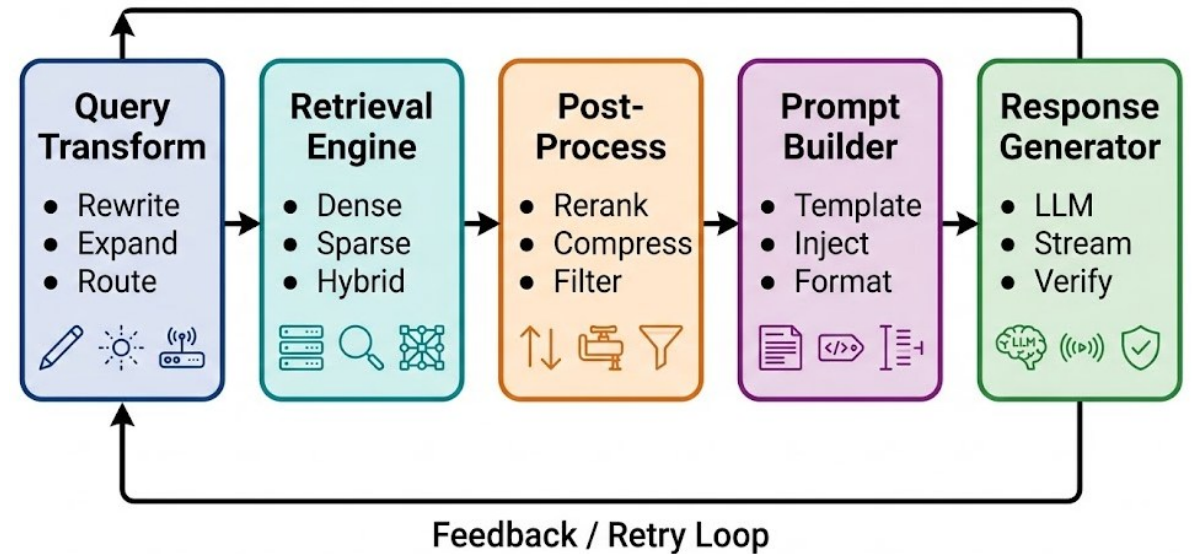
MODULAR RAG

Modular RAG breaks the RAG pipeline into **independent, swappable modules** with well-defined interfaces
Each module can be developed, tested, and upgraded independently

Key characteristics:

- Simple to implement (< 50 lines of code)
- Each module has a standardised input/output contract
- Modules can be mixed and matched (e.g., swap FAISS for Qdrant without touching the rest)
- Supports routing different query types can take different paths through the pipeline
- Enables A/B testing of individual components

Best for: Teams building RAG as a product with evolving requirements and multiple retrieval strategies



AGENTIC RAG

Agentic RAG replaces the fixed pipeline with an **autonomous agent** that dynamically decides what to do at each step

The agent can:

- Retrieve
- Re-retrieve if results are not enough (self-corrective RAG)
- Ask clarifying questions
- Create subqueries for complex queries
- Call external tools and combine information
- Combine multiple strategies based on the query

Best for: Complex, multi-step questions that span multiple data sources; research assistants; enterprise knowledge systems where query types are highly diverse

However, agentic RAG pipelines have higher latency and cost, and are less predictable



GRAPH RAG

Graph RAG augments vector retrieval with a knowledge graph to capture relationships between entities that flat chunk retrieval misses

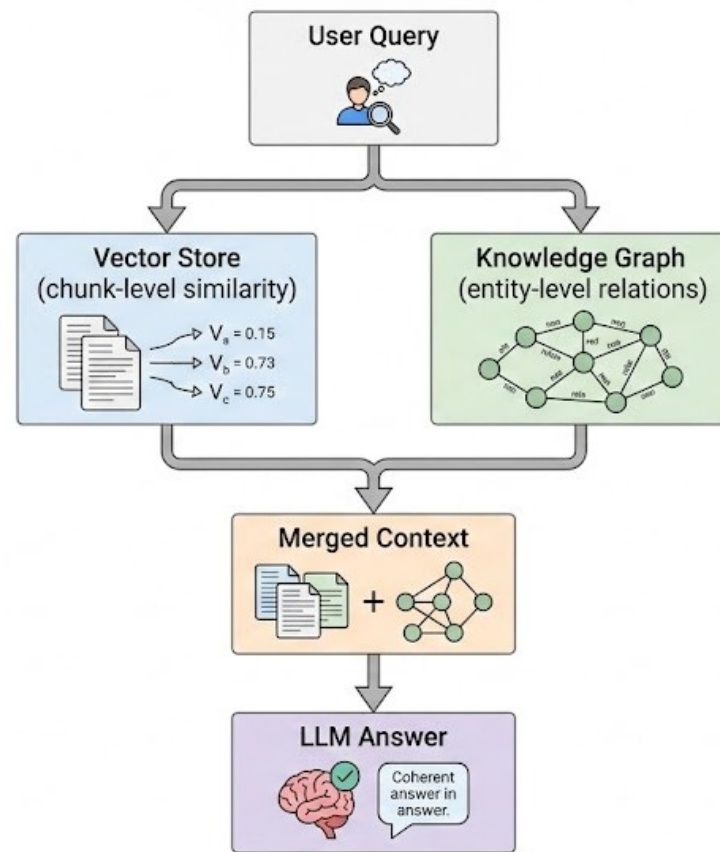
Standard RAG retrieves isolated chunks,
but some questions require understanding relationships:

"Who reports to the VP of Engineering?"

or

"What products use the same supplier as Product X?"

A knowledge graph stores entities and their typed relationships explicitly



Graph RAG System Overview

GRAPH RAG

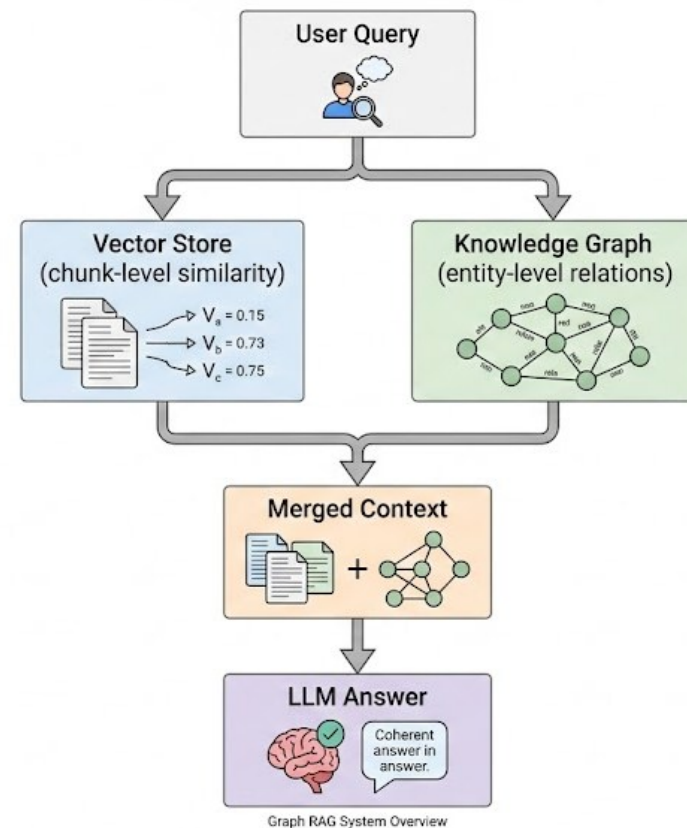
How it works:

- During indexing, extract entities and relationships from documents (using NLP or an LLM) and build a graph
- During retrieval, traverse the graph to find connected entities, then retrieve the associated text chunks
- The LLM receives both the graph-derived context and the vector-retrieved chunks

Best for: Domains with rich relational structure: organisational hierarchies, supply chains, biomedical knowledge, legal case references

One of the biggest strengths is “multi-hop” retrieval, pulling related chunks that are logically related for a query instead of just singular retrieval

Suggestion: Go for graph rags for specialised use cases where naïve or vector rags are not successful



COMPARISON

Framework	Complexity	Latency	Flexibility	Best For
Naive RAG	Low	Low	Low	Prototypes, clean single-domain corpora
Advanced RAG	Medium	Medium	Medium	Production Q&A with quality requirements
Modular RAG	Medium-High	Medium	High	Teams building RAG as a product
Agentic RAG	High	High	Very High	Multi-source, complex, multi-step queries
Graph RAG	High	Medium-High	Medium	Relationship-heavy, structured domains

NEED FOR CONTEXT COMPRESSION

Every LLM has a finite context window: a hard ceiling on how many tokens it can process in a single call

Even models with 128K or 200K context windows are constrained by:

- **Cost:** LLM pricing is per-token. More context = higher cost per query.
- **Latency:** Processing more tokens takes longer. Time-to-first-token increases roughly linearly with prompt length.
- **Lost in the middle:** Research (Liu et al., 2023) shows that LLMs pay the most attention to the beginning and end of the context and tend to ignore information in the middle. More chunks amplify this problem.
- **Signal dilution:** If 5 out of 20 retrieved chunks are relevant, the 15 irrelevant ones compete for the model's attention and can degrade answer quality.

The goal of compression is to **maximise the information density** of the context passed to the LLM: more signal, fewer tokens, less noise

TOKEN-BUDGET CONSTRAINTS

A practical way to think about compression is through the lens of a **token budget**:

Total context window:	128,000 tokens	
└─ System prompt:	200 tokens	
└─ User query:	50 tokens	
└─ Reserved for response:	2,000 tokens	
└─ Available for context:	125,750 tokens	← estimated budget
But cost-optimal budget:	~4,000-8,000 tokens	

Even if the window allows 125K tokens of context, the cost-optimal and quality-optimal budget is much smaller

Compression lets us retrieve broadly (for recall) and then compress aggressively (for precision and cost)

Let's see some compression techniques

RELEVANCE FILTERING

The simplest form of compression: **drop chunks below a similarity threshold**

```
Retrieved chunks (by similarity score):  
Chunk 1: 0.92 – highly relevant      ✓ Keep  
Chunk 2: 0.87 – relevant              ✓ Keep  
Chunk 3: 0.74 – somewhat relevant    ✓ Keep  
Chunk 4: 0.61 – marginally relevant  ✗ Drop  
Chunk 5: 0.45 – not relevant          ✗ Drop
```

Set a threshold (e.g., 0.70) and only pass chunks that clear it

This is cheap (no LLM call needed) and effective at removing obvious noise

Variants:

- **Fixed threshold:** Drop everything below score X
- **Dynamic threshold:** Drop everything below ($\text{top_score} \times 0.8$); adapts to query difficulty
- **Diversity filtering:** Among similarly scored chunks, prefer those that cover different subtopics such as using Maximal Marginal Relevance (MMR)

EXTRACTIVE COMPRESSION

Instead of keeping or dropping entire chunks, **extract only the relevant sentences or spans** from each chunk

Original chunk (180 tokens):

"The company was founded in 2005 in San Francisco.
It started as a consulting firm before pivoting to SaaS.
In Q4 2024, revenue reached \$1.2B, up 18% YoY. ← relevant
The office has a dog-friendly policy.
Cloud services drove most of the growth. ← relevant
The cafeteria serves organic meals."

Compressed chunk (30 tokens):

"In Q4 2024, revenue reached \$1.2B, up 18% YoY.
Cloud services drove most of the growth."

This can be done with:

- **LLM-based extraction:** Ask an LLM to extract only query-relevant sentences from each chunk
- **Embedding-based extraction:** Embed each sentence within the chunk, keep only those above a similarity threshold to the query
- **Attention-based extraction:** Use a small specialised model (like Long LLMingua) that identifies which tokens the generator would attend to most

ABSTRACTIVE COMPRESSION

Instead of extracting verbatim spans, **summarise** the retrieved content into a condensed form.

```
5 retrieved chunks (total: 1,200 tokens)
  → LLM summarisation prompt
  → Compressed summary (200 tokens)
```

Pros: Can compress aggressively (more than 10x); produces coherent, deduplicated text

Cons: Requires an additional LLM call (adds latency and cost); the summariser may drop important details or introduce errors

When to use: When you retrieve many chunks that overlap significantly (e.g., multiple news articles about the same event)

CONTEXTUAL CHUNKING

Rather than compressing at query time, **enrich chunks at index time** with a **contextual summary** that makes each chunk self-contained (compression at index time)

Original chunk:

"Revenue grew 18% YoY." ← lacks context (which company? which period?)

Contextually enriched chunk:

"In Q4 2024, Acme Corp's revenue grew 18% year-over-year to \$1.2B, driven by the cloud services segment. [Source: Q4 2024 Earnings Report]"

The enriched chunk is slightly longer but carries more information per token, reducing the need for retrieving multiple chunks to assemble context

RECALL vs PRECISION TRADE-OFF

Compression is fundamentally a bet: you trade **recall** (having all potentially useful information) for **precision** (having only the most useful information in fewer tokens)

More aggressive compression:

- ✓ Lower cost
- ✓ Lower latency
- ✓ Less "lost in the middle"
- ✗ Risk dropping a critical fact

Less aggressive compression:

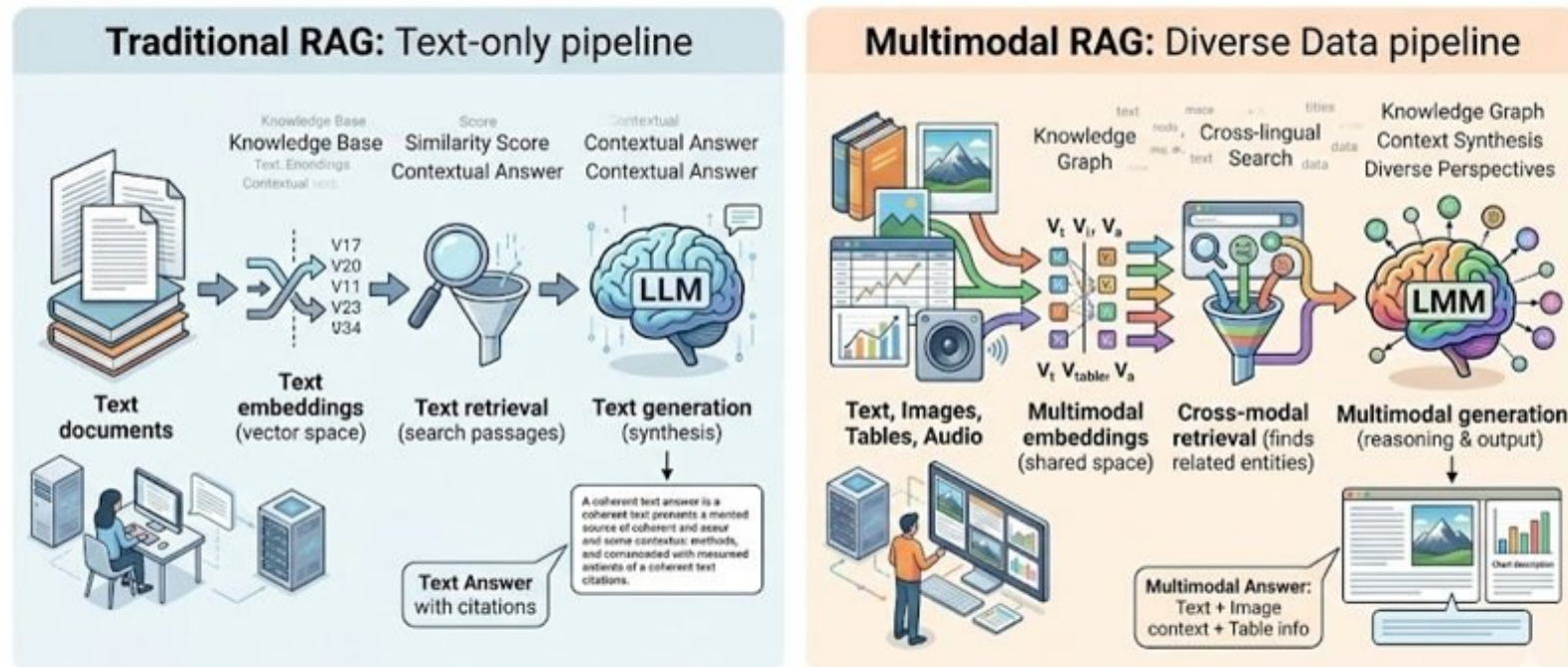
- ✓ All relevant info retained
- ✗ Higher cost
- ✗ Higher latency
- ✗ More noise for the LLM to filter

- Start with relevance filtering (cheap, safe) and add extractive or abstractive compression only if needed
- Evaluate context utilisation if the LLM's answer consistently uses only 2 of 10 provided chunks, you are over-retrieving and should compress more
- For high-stakes domains (medical, legal), err on the side of less compression; A missed fact is more costly than a slightly longer latency

BEYOND TEXT

The real world is not text-only; Enterprise documents contain **tables, charts, diagrams, photographs, code screenshots, and scanned PDFs**

A text-only RAG system is blind to all of this Multimodal RAG extends the pipeline to retrieve and reason over multiple modalities



MULTIMODAL EMBEDDINGS

We already know, the foundation of multimodal RAG is a **shared embedding space** where different modalities can be compared directly

If an image of a dog and the text "a photo of a dog" are embedded into the same vector space, semantic search works across modalities

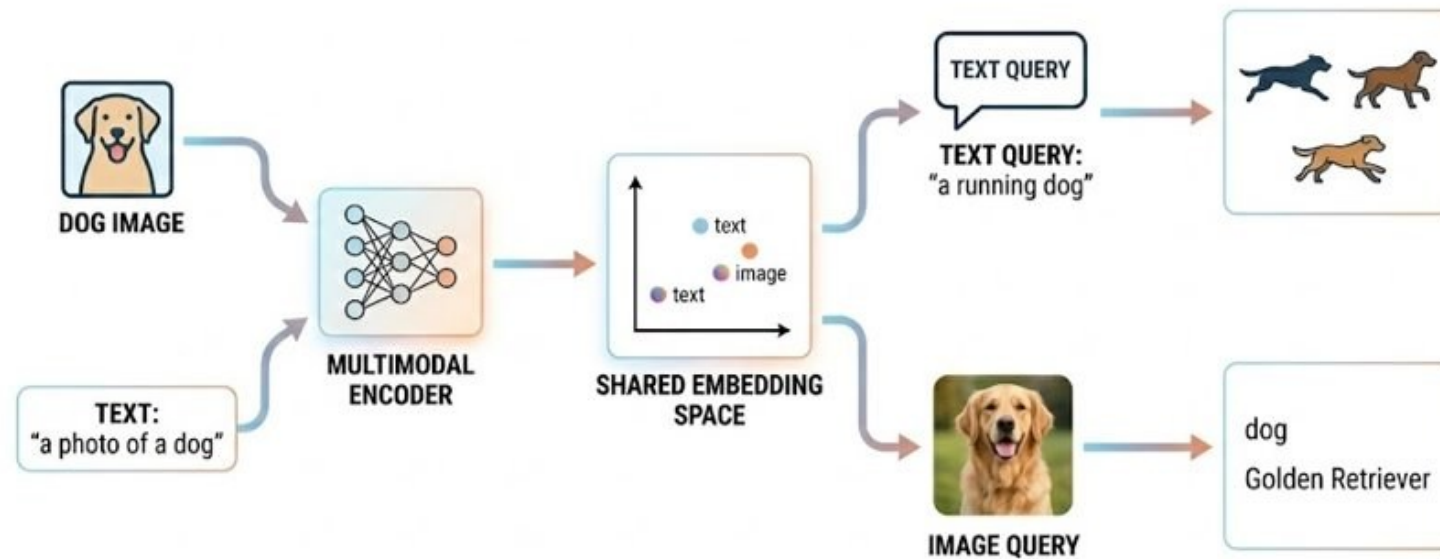
Key models:

Model	Modalities	Approach
CLIP (OpenAI)	Text + Image	Contrastive learning aligns text and image embeddings
BLIP-2 (Salesforce)	Text + Image	Uses a Q-Former to bridge frozen image and text encoders
ImageBind (Meta)	Text + Image + Audio + Depth + Thermal + IMU	Binds 6 modalities into a single space using image as the anchor
Nomic Embed Vision	Text + Image	Open-source, unified text-image embeddings
ColPali	Text + Document Images	Treats entire document pages as images; embeds them directly without OCR

MULTIMODAL QUERYING

How they work:

These models are trained with contrastive objectives as they learn to pull matched pairs (an image and its caption) closer in embedding space and push unmatched pairs apart



MULTI-MODAL RAG APPROACHES

1. Convert Everything to Text

- The simplest strategy is to convert all non-text content into text before indexing
- **Pros:** Reuses your entire text RAG pipeline unchanged
- **Cons:** A caption can never fully describe a complex chart; Spatial relationships, visual patterns, and layout information are lost

MULTI-MODAL RAG APPROACHES

2. Native Multimodal Embeddings

- Embed text and images (or other modalities) into a shared vector space using models like CLIP
- A text query can retrieve images directly
- **Pros:** True cross-modal retrieval; no information loss from captioning
- **Cons:** Text-image alignment is imperfect for domain-specific content; embedding models may not handle specialised visuals (medical scans, engineering diagrams) well without fine-tuning

MULTI-MODAL RAG APPROACHES

3. Late Fusion (Separate Retrieval, Combined Generation)

- Maintain **separate indices** for each modality
- Retrieve from each independently, then pass all results to a multimodal LLM (like GPT-4o) for generation
- **Pros:** Each modality uses its best retriever; the multimodal LLM handles cross-modal reasoning
- **Cons:** More complex infrastructure; requires a multimodal LLM for generation

CHALLENGES IN MULTI-MODAL RAG

Challenge	Description
Alignment quality	Generic CLIP-style models struggle with domain-specific visuals. A chart about "Q4 revenue" may not embed near the text "Q4 revenue" if the model hasn't seen financial charts.
Grounding hallucinations	Multimodal LLMs can hallucinate about image content (e.g., reading numbers from a chart incorrectly). Verifying visual claims is harder than verifying text claims.
Chunking visual content	How do you "chunk" an image? A full-page screenshot, a cropped chart, a single table; the granularity of visual chunks is less well-defined than text.
Token cost	Images consume many tokens when passed to multimodal LLMs (a single image can cost 500-2000+ tokens). This shrinks the budget for text context.
Evaluation	Measuring retrieval quality for images is harder, there's no equivalent of "exact match" or "F1" for cross-modal retrieval
Latency	Image embedding and multimodal generation are slower than text-only pipelines

WHEN TO USE MULTI-MODAL RAG

Scenario	Multimodal RAG Needed?
Q&A over text-only documents (policies, FAQs)	No – standard text RAG is sufficient
Documents with important charts, graphs, diagrams	Yes – text-only RAG misses visual information
Scanned PDFs with no selectable text	Yes – need OCR or direct image embedding
Product catalogues with images	Yes – users search by visual similarity
Audio/video content (podcasts, meetings)	Yes – transcribe and/or embed audio directly
Code repositories with screenshots of UIs	Yes – pair screenshots with code for UI-grounded answers

CACHING

Every RAG query involves at least two expensive operations:

- **Retrieval:** Embed the query + vector search (fast, but not free)
- **Generation:** LLM inference (slow and expensive)

In many production systems, queries are **repetitive**: the same question (or nearly the same question) is asked repeatedly, each repetition pays the full cost again

Caching: We can store frequently accessed components (like embeddings, retrieved document chunks, or final LLM responses) to minimise latency and lower API costs

Without caching:

```
Query 1: "What is the return policy?" → embed + search + LLM → 2.1s, $0.003
Query 2: "What is the return policy?" → embed + search + LLM → 2.1s, $0.003
Query 3: "What's your return policy?" → embed + search + LLM → 2.1s, $0.003
Total: 6.3s, $0.009
```

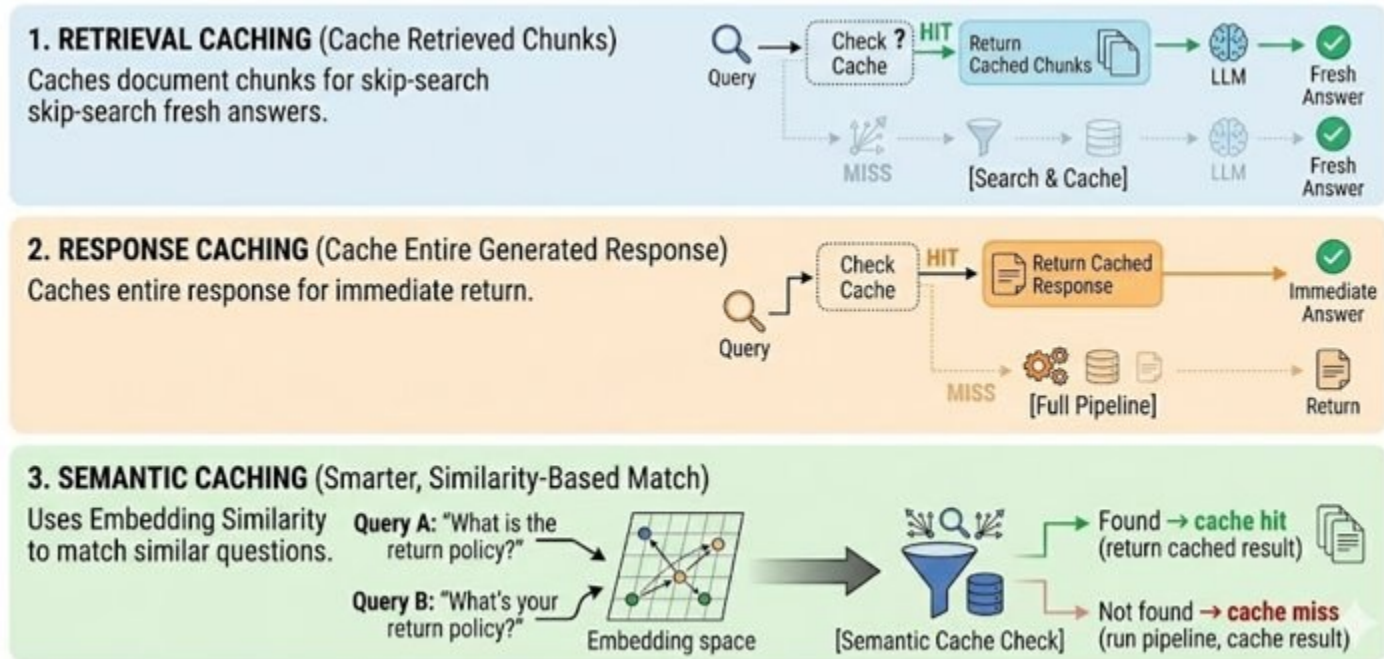
With caching:

```
Query 1: "What is the return policy?" → embed + search + LLM → 2.1s, $0.003
Query 2: "What is the return policy?" → cache hit → 5ms, $0.000
Query 3: "What's your return policy?" → semantic cache hit → 15ms, $0.000
Total: 2.1s, $0.003 (67% cheaper, 67% faster)
```


TYPES OF CACHING

There are different types of caching that can be applied in a RAG system:

- Retrieval Caching
- Response Caching
- Semantic Caching



RETRIEVAL CACHING

Cache the **retrieved chunks** for a given query (or query embedding)

If the same or similar query arrives again, skip the vector search and return the cached chunks;
Generation still runs

Query → [Check retrieval cache]

└─ HIT: Return cached chunks → LLM → Answer

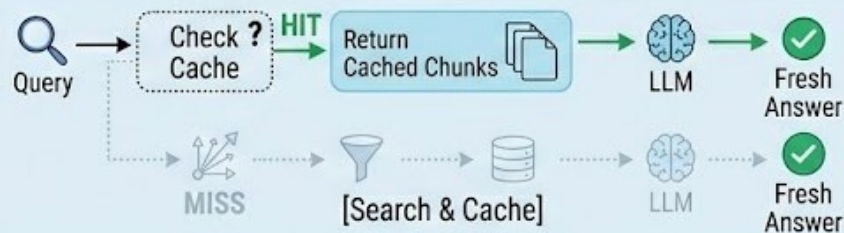
└─ MISS: Embed → Search → Cache results → LLM → Answer

Pros: Saves retrieval latency and cost; generation still runs, so the answer reflects the latest prompt/model

Cons: LLM cost is not saved; cache invalidation is needed when the document store is updated

1. RETRIEVAL CACHING (Cache Retrieved Chunks)

Caches document chunks for skip-search
skip-search fresh answers.



RESPONSE CACHING

Cache the **entire generated response** for a given query

If the same query arrives again, return the cached response immediately, no retrieval, no generation

Query → [Check response cache]

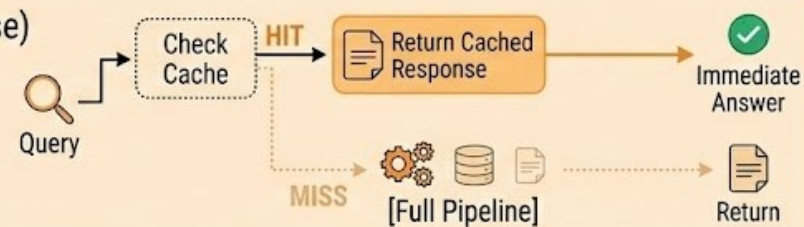
- └ HIT: Return cached response (no retrieval, no LLM)
- └ MISS: Full RAG pipeline → Cache response → Return

Pros: Maximum savings as it eliminates both retrieval and generation cost/latency

Cons: Cached responses may become stale when the underlying documents change; requires careful invalidation

2. RESPONSE CACHING (Cache Entire Generated Response)

Caches entire response for immediate return.



SEMANTIC CACHING

A smarter variant: instead of exact-match cache keys, use **embedding similarity** to match queries

"What is the return policy?" and "What's your return policy?" are different strings but the same question and a semantic cache recognises this

Cache lookup:

1. Embed the incoming query
2. Search the cache index for embeddings within a similarity threshold
3. If found → cache hit (return cached result)
4. If not found → cache miss (run pipeline, cache the result)

Implementation: Use a small vector store (or even a FAISS index) as the cache index

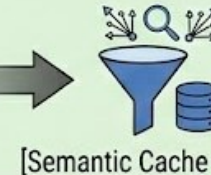
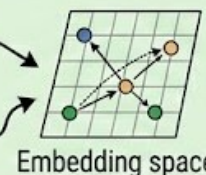
Each entry stores the query embedding, the cached response (or cached chunks), and a TTL

3. SEMANTIC CACHING (Smarter, Similarity-Based Match)

Uses Embedding Similarity to match similar questions.

Query A: "What is the return policy?"

Query B: "What's your return policy?"



Found → **cache hit**
(return cached result)

Not found → **cache miss**
(run pipeline, cache result)

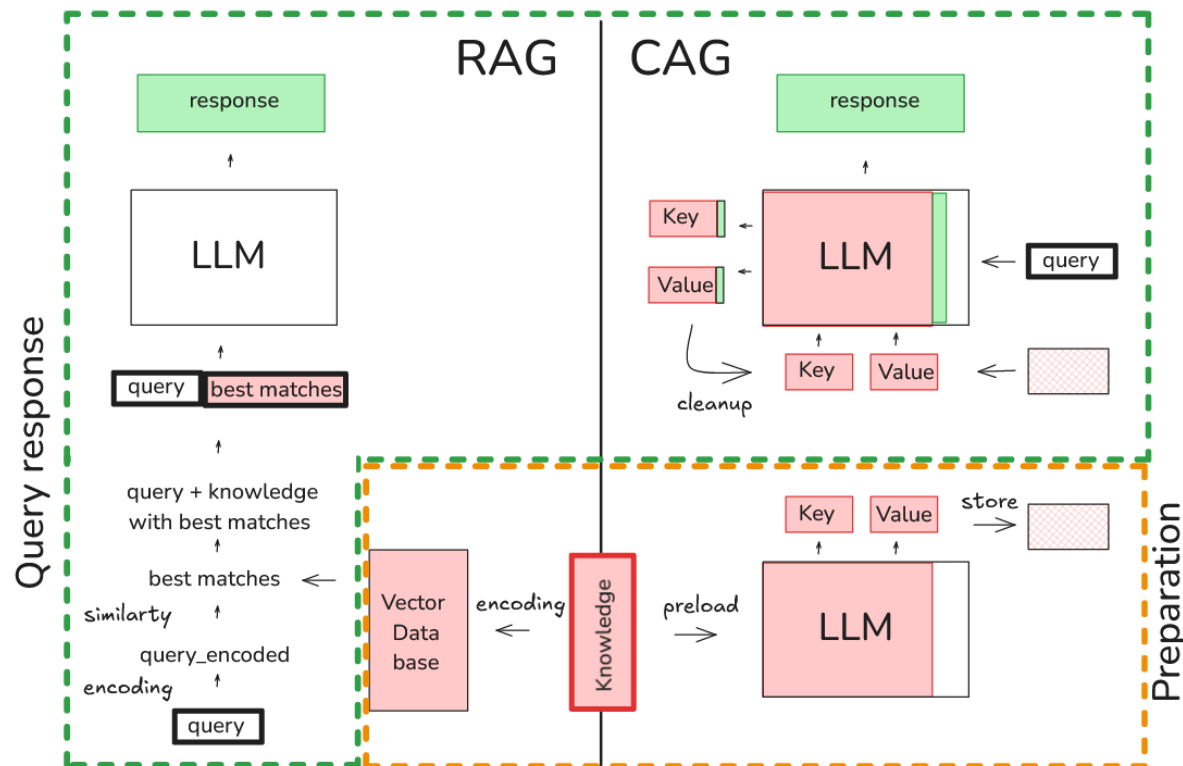


CACHE-AUGMENTED GENERATION (CAG)

The aforementioned caching techniques are used as a part of RAG pipeline, they operate outside the model

Cache-Augmented Generation is a more advanced approach

Instead of retrieving documents per-query, **pre-load the entire relevant knowledge base into the LLM's context (or KV cache) once**, and then answer multiple queries against that cached context



KEY-VALUE CACHE

Naïve generation is inefficient

During autoregressive generation, the model recomputes attention over all previous tokens to create each new token

Input: "The capital of France is.."

Cost grows as with increasing input tokens

Time complexity $\approx O(n^2)$: Redundant computation at every step

The Key-Value Cache is a mechanism that stores attention keys and values from previous tokens and reuses them during decoding. In self-attention, each token produces Keys (K) and Values (V). The KV cache stores these.

1. For the initial prompt: compute K, V for all tokens and store in cache
2. For (only) new tokens: compute K, V; attend to cached K, V, and current token
3. Append new K, V to the cache

CAG MECHANISM

How it works:

The entire knowledge base (or a relevant subset) is pre-loaded into the LLM's extended context window or KV cache
The KV cache is serialised and stored (e.g., to disk or GPU memory)

For each new query, the cached KV state is restored, and the query is appended so that the model can attend to all pre-loaded documents without re-encoding them

When CAG is effective:

Condition	CAG Suitable?	Why
Small, stable knowledge base (< 100K tokens)	Yes	Fits in context window; rare updates mean low invalidation cost
Queries are diverse but all target the same corpus	Yes	Amortises the one-time loading cost across many queries
Knowledge base changes frequently	No	Every update invalidates the cache and requires re-loading
Very large corpus (millions of chunks)	No	Cannot fit in context; traditional retrieval is necessary
Low-latency requirements with repetitive queries	Yes	Eliminates retrieval entirely after the initial load

OTHER OPTIMISATION TECHNIQUES

Beyond caching, several techniques improve RAG system performance:

1. Embedding Pre-computation and Batching

- **Pre-compute** all document embeddings offline. Never embed documents at query time
- **Batch** query embeddings when possible (e.g., if processing a queue of queries)
- Use **quantised embeddings** (float16 or int8) to reduce storage and speed up similarity computation with minimal accuracy loss

2. Streaming Generation

- Instead of waiting for the full LLM response before returning to the user, **stream tokens** as they are generated
- This dramatically improves perceived latency, the user sees the answer forming in real time, even if total generation time is unchanged

3. Async and Parallel Retrieval

- When using hybrid search or multi-index retrieval, run the dense and sparse legs **in parallel** rather than sequentially
- Similarly, if using re-ranking, start generating with the bi-encoder results while the cross-encoder re-ranks; if the re-ranker produces a significantly different ordering, regenerate

VECTOR INDEX OPTIMISATION TECHNIQUES

A vector store supports **approximate nearest neighbour (ANN) search** over embeddings

Core bottlenecks:

- High-dimensional similarity search
- Large corpus size (millions–billions of vectors)
- Real-time latency constraints

Technique	What It Does	Trade-off
HNSW (Hierarchical Navigable Small World)	Approximate nearest neighbour index; logarithmic search time (graph-based)	Small accuracy loss for massive speed gain
IVF (Inverted File Index)	Partitions the vector space into clusters; searches only relevant clusters	Must tune <i>nprobe</i> (clusters to search)
Product Quantisation	Compresses vectors to reduce memory footprint (clusters of sub-vectors)	Larger accuracy loss, low recall; best for very large corpora
Scalar Quantisation	Reduces precision of vectors (<i>float32</i> → <i>int8</i>)	Minimal accuracy loss; significant memory savings